

Kolorowanie Grafu

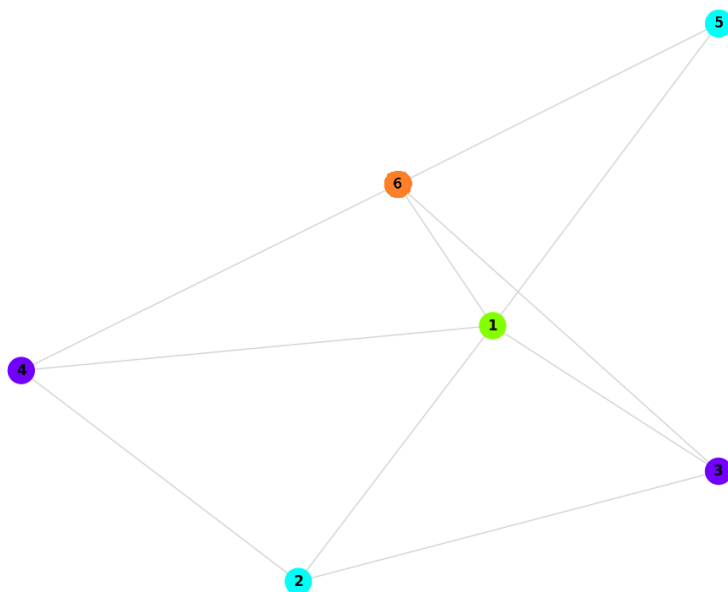
Adam Woźniński 164119
Marcin Jakubowski 164108

Styczeń 2026

Algorytm Tabu Search

1 Inicjalizacja

Inicjalizacja dla instancji początkowej (6 wierzchołków około 64% gęstości) pokolorowanej algorytmem zachłannym



2 Opis Algorytmu oraz Pseudokod

Algorytm działa w trybie iteracyjnym, próbując w każdej głównej pętli zmniejszyć liczbę użytych kolorów o jeden względem najlepszego dotychczasowego rozwiązania.

1. Algorytm rozpoczyna działanie od wygenerowania rozwiązania początkowego za pomocą algorytmu zachłannego (**greedy_coloring**). Następnie wchodzi w pętlę główną, kontrolowaną przez limit czasu **t**.

Kluczowym elementem strategii jest funkcja **tabu_initialize**. Jej zadaniem jest „wymuszenie” poszukiwania rozwiązania lepszego niż obecne. Funkcja ta pobiera najlepsze znane rozwiązanie (**colored_graph**), identyfikuje najwyższy użyty numer koloru (**max_color**), a następnie stawia za cel znalezienie kolorowania używającego **max_color - 1** barw.

Aby to osiągnąć, algorytm identyfikuje wszystkie wierzchołki pokolorowane kolorem maksymalnym (**targets_list**) i zmienia ich kolor na inny, wybierając taki, który generuje najmniejszą liczbę konfliktów z sąsiadami (**min_from_neighbours_colors**). Ten proces celowo wprowadza konflikty (krawędzie łączące wierzchołki o tym samym kolorze), które algorytm będzie starał się naprawić w kolejnej fazie.

2. Faza Naprawcza i Przeszukiwanie Sąsiedztwa

Po inicjalizacji następuje właściwe przeszukiwanie Tabu (pętla wewnętrzna), która trwa dopóki istnieją konflikty w grafie (**conflicts_list**). W każdym kroku tej pętli:

- Algorytm losowo wybiera jeden wierzchołek, który generuje konflikt (**conflict_node = random.choice**).
- Dla wybranego wierzchołka sprawdzane są wszystkie możliwe kolory z dostępnego zakresu (od 1 do **max_color**).
- Dla każdego potencjalnego koloru obliczany jest koszt ruchu, definiowany jako liczba konfliktów, które powstałyby po zmianie barwy wężła (**conflicts_for_node**).

3. Mechanizm Tabu i Kryterium Aspiracji

Aby uniknąć zapętlenia się algorytmu i powrotu do niedawno odwiedzonych stanów, stosowana jest lista zabronionych ruchów (**tabu**). Lista ta przechowuje pary: (**wierzchołek, poprzedni_kolor**). Oznacza to, że jeśli zmieniamy kolor wężła z A na B, to przez pewien czas powrót do koloru A jest zabroniony. Lista działa jak kolejka FIFO o ustalonej długości (**max_lenght**).

W kodzie zaimplementowano tzw. **kryterium aspiracji**. Jest ono widoczne w warunku sprawdzającym listę tabu: ruch jest blokowany tylko wtedy, gdy znajduje się na liście tabu **oraz** liczba konfliktów po jego wykonaniu jest większa od zera. Oznacza to, że jeśli dany ruch całkowicie eliminuje konflikty dla danego wężła (prowadzi do rozwiązania dopuszczalnego lokalnie), jest on dozwolony nawet jeśli znajduje się na liście tabu.

4. Wybór Ruchu i Restart

Spośród dozwolonych ruchów wybierany jest ten, który minimalizuje liczbę konfliktów (**min_conflict_val**). W przypadku remisów (wielu kandydatów o tym samym, minimalnym koszcie), wybór następuje losowo. Po wykonaniu ruchu aktualizowana jest tablica rozwiązań (**temp_solution**), a wykonany ruch trafia na początek listy tabu.

Algorytm posiada również zabezpieczenie przed stagnacją. Jeśli w ciągu ostatnich 30 sekund nie udało się znaleźć poprawnego rozwiązania (brak usunięcia wszystkich konfliktów), pętla wewnętrzna jest przerywana (warunek **time.time() - last_improvement_time > 30**). Powoduje to powrót do pętli głównej i ponowną inicjalizację procesu.

```

TabuSearch(Graf, CzasDziałania, DługośćListyTabu);
begin
    CzasStart ← PobierzCzas();
    NajlepszeRozwiązanie, _ ← AlgorytmZachłanny(Graf);
    CzasOstatniejPoprawy ← CzasStart;

    while PobierzCzas() - CzasStart < CzasDziałania do
        // Próbuje się zejść z liczbą kolorów o 1 w dół
        RozwiązanieTymczasowe, MaxKolor ← TabuInitialize(Graf,
            NajlepszeRozwiązanie);
        ListaTabu ← ∅;
        ListaKonfliktów ← ZnajdźWszystkieKonflikty(Graf,
            RozwiązanieTymczasowe);

        while ListaKonfliktów ≠ ∅ oraz Czas nie minął do
            // Mechanizm restartu (brak poprawy przez 30s)
            if PobierzCzas() - CzasOstatniejPoprawy > 30 sekund then
                | przerwij PętlęWewnętrzną;

            WęzełKonfliktowy ← LosowyWybór(ListaKonfliktów);
            KosztyRuchów ← Tablica wypełniona ∞;

            // Sprawdź każdy możliwy kolor dla wybranego węzła
            for Kolor ← 1 do MaxKolor do
                SymulacjaRozwiązania ← Kopia(RozwiązanieTymczasowe);
                SymulacjaRozwiązania[WęzełKonfliktowy] ← Kolor;
                LiczbaKonfliktów ← ObliczKonflikty(Graf, Węzeł, Symulacja);
                KosztyRuchów[Kolor] ← LiczbaKonfliktów;

            // Filtracja TABU i Kryterium Aspiracji
            foreach (Kolor, Koszt) w KosztyRuchów do
                Ruch ← (WęzełKonfliktowy, Kolor);
                if (Ruch ∈ ListaTabu) oraz (Koszt > 0) then
                    | KosztyRuchów[Kolor] ← ∞; // Zablokuj ruch

            MinKoszt ← min(KosztyRuchów);
            if MinKoszt == ∞ then
                | kontynuuj Pętlę (Brak ruchów);

            Kandydaci ← Lista kolorów z kosztem MinKoszt;
            StaryKolor ← RozwiązanieTymczasowe[WęzełKonfliktowy];
            WybranyNowyKolor ← LosowyWybór(Kandydaci);
            RozwiązanieTymczasowe[WęzełKonfliktowy] ← WybranyNowyKolor;
            // Aktualizacja listy Tabu
            WstawNaPoczątek(ListaTabu, (WęzełKonfliktowy, StaryKolor));
            if |ListaTabu| > DługośćListyTabu then
                | UsuńOstatniElement(ListaTabu);
            ListaKonfliktów ← ZnajdźWszystkieKonflikty(Graf,
                RozwiązanieTymczasowe);

            if ListaKonfliktów jest Pusta then
                | NajlepszeRozwiązanie ← Kopia(RozwiązanieTymczasowe);
                | CzasOstatniejPoprawy ← PobierzCzas();

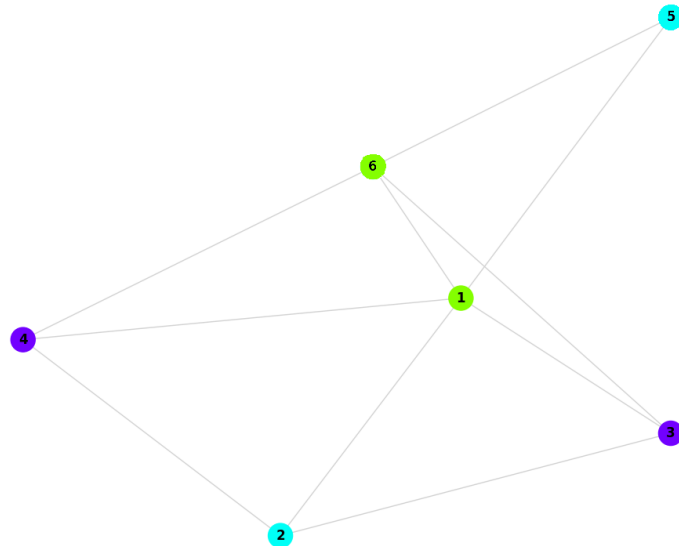
return NajlepszeRozwiązanie, max(NajlepszeRozwiązanie)

```

3 Przykład obrazujący działanie

3.1 Krok 1 Inicjalizacja Tabu Search

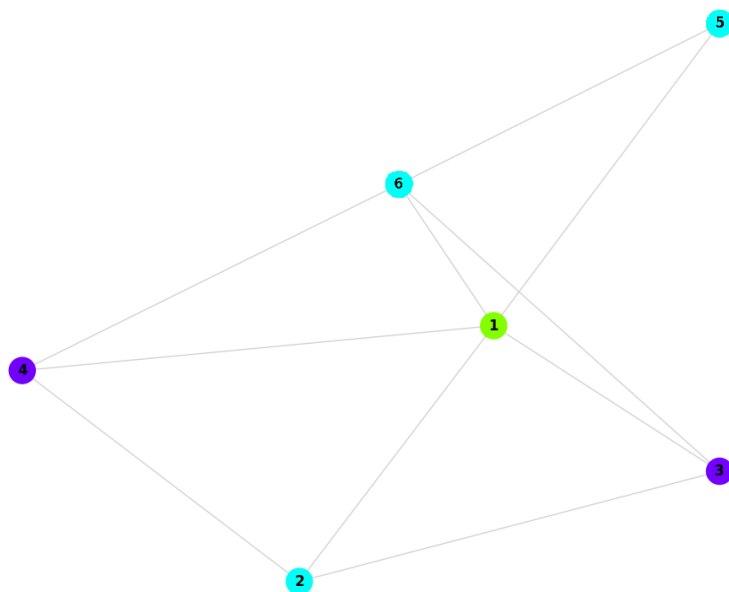
Algorytm wymusza redukcję liczby kolorów. Musimy pozbyć się Koloru 4. Jedynym węzłem z tym kolorem jest Wierzchołek 6. Musimy zmienić jego kolor na jeden z zakresu 1, 2, 3, nawet jeśli spowoduje to konflikt. Algorytm wybiera losowo jeden z ruchów o najmniejszym koszcie. Wybierzmy zmianę na Kolor 1. Lista Tabu: Pusta.



3.2 Krok 2: Pierwsza Iteracja – Rozwiązywanie konfliktu (1, 6)

Mamy konflikt między węzłem 1 a 6. Algorytm wybiera jeden z nich do naprawy. Wybiera łatwiejszy węzeł: Węzeł 6. Wybieramy kolor o najmniejszej liczbie konfliktów Kolor 2.

Lista Tabu: [(6,1)].



3.3 Krok 3: Druga Iteracja – Rozwiązywanie konfliktu (5, 6)

Konflikt jest teraz między węzłami 5 i 6. Wybieramy węzeł do zmiany, padło na Węzeł 5. Wybieramy kolor o najmniejszej liczbie konfliktów Kolor 3. Lista Tabu: [(5,2), (6,1)]

3.4 Krok 4: Zakończenie i Weryfikacja

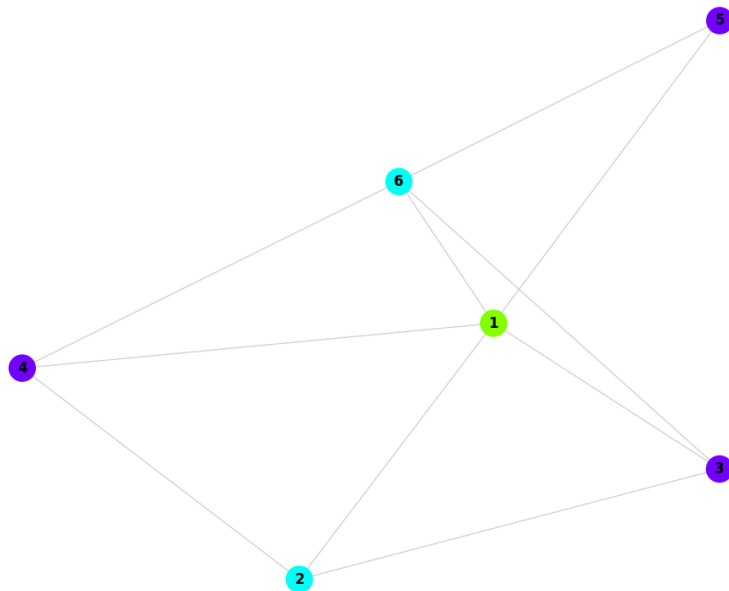
Algorytm sprawdza listę konfliktów. Jest pusta. Udało się zejść z 4 kolorów na 3. Algorytm kończy pracę z sukcesem po 2 iteracjach naprawczych.

4 Finalizacja

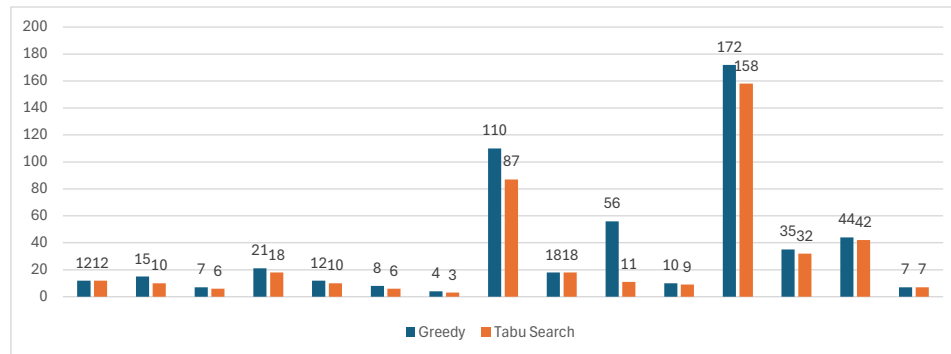
Ostateczna lista kolorów prezentuje się następująco:

[(1, 1), (2, 2), (3, 3), (4, 3), (5, 3), (6, 2), 3]

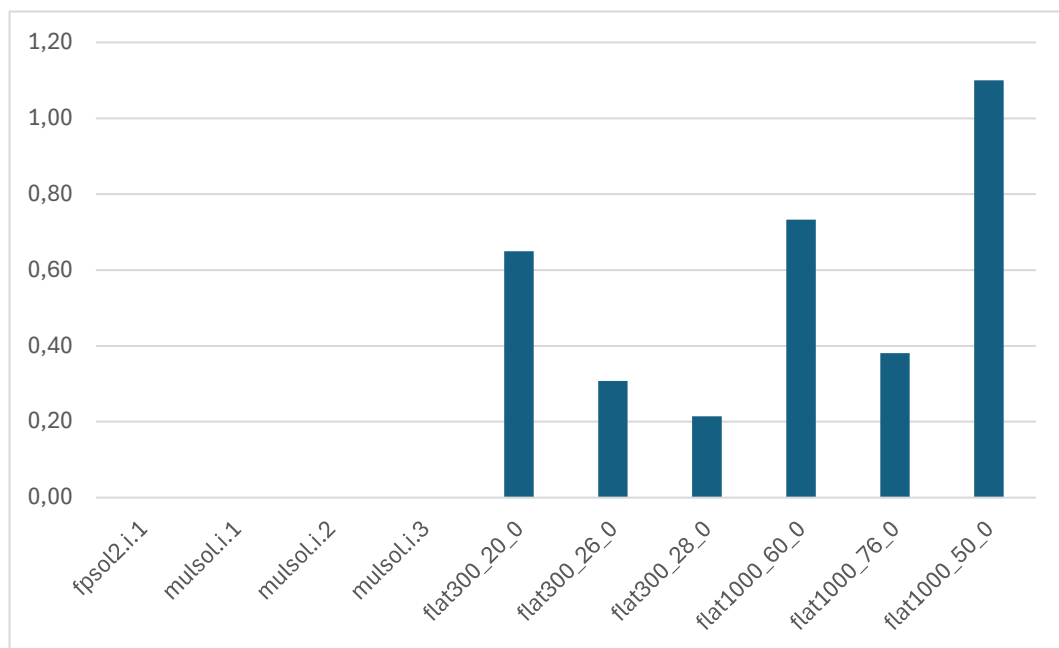
Gdzie pierwsza liczba w nawiasie to wierzchołek a druga liczba to numer koloru. Ostatnia liczba oznacza liczbę kolorów w grafie.



5 Wykresy



Rysunek 1: Porównanie Tabu Search do algorytmu zachłannego



Rysunek 2: Wartość błędu względnego Tabu Search w stosunku do wartości optymalnej.

Tabela 1: Wyniki algorytmu Tabu Search dla instancji rankingowych

Instancja	Wynik (Tabu)
gc_1000	137
gc500	72
le450_5a	6
miles250	8
queen6	7